

The $\sqrt{d}$ Scaling Factor: From Variance to Transformers

A Bridge Document for the Proust Attention Machine

Proust Attention Machine Project

February 7, 2026

Abstract

This document answers one question: *why does $\sqrt{d}$ appear everywhere in transformers?* We build the answer from first principles — starting with the variance of a single random variable, then sums, then dot products, and finally connecting to both the $1/\sqrt{d_k}$ scaling in attention and the $\sqrt{d_{\text{model}}}$ scaling in embeddings. Every step uses concrete numerical examples from our 64-dimensional Proust model. Three layers of explanation are used throughout: formal definitions (blue), geometric intuition (green), and practical application to our code-base (orange).

Contents

1	Notation and Conventions	2
2	Prerequisite: What Is Variance?	2
3	The Core Theorem: Variance of a Sum	3
4	Dot Products: Where $\sqrt{d}$ Becomes Critical	4
5	Why Softmax Cares About Scale	5
6	The Other $\sqrt{d}$: Embedding Scaling	6
7	Unifying the Two Scalings	8
8	The Deeper Connection: L2 Norm and Dimension	9
9	Summary Table	10
10	Key Identities and Properties	11
11	What This Document Does Not Cover (Yet)	11

1 Notation and Conventions

Symbol	Meaning	Our value
d	Generic dimension (number of elements in a vector)	varies
d_{model}	Embedding dimension	64
d_k	Per-head dimension ($d_{\text{model}}/n_{\text{heads}}$)	32
n_{heads}	Number of attention heads	2
σ	Standard deviation of a random variable	varies
σ^2	Variance of a random variable	varies
$\mathbb{E}[X]$	Expected value (average) of random variable X	—
$\text{Var}(X)$	Variance of random variable X	—
$\mathbf{q}, \mathbf{k}$	Query and key vectors in attention	$(d_k,)$

We write vectors in bold ($\mathbf{v}$) and scalars in italics (x). Shape annotations appear in comments: (batch, seq, d_model).

2 Prerequisite: What Is Variance?

Before we can understand $\sqrt{d}$, we need to be precise about variance.

Definition: Variance

The **variance** of a random variable X measures how spread out its values are from the mean:

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$$

The **standard deviation** is $\sigma = \sqrt{\text{Var}(X)}$.

A random variable $X \sim \mathcal{N}(0, 1)$ (standard normal) has:

$$\mathbb{E}[X] = 0, \quad \text{Var}(X) = 1, \quad \sigma = 1$$

This means values are typically between -2 and $+2$ (within 2 standard deviations).

What variance feels like

Variance tells you **how big the numbers tend to be** (ignoring sign).

- Variance = 1: numbers are typically around ± 1 . Like a gentle wave.
- Variance = 64: numbers are typically around ± 8 ($\sigma = \sqrt{64} = 8$). Much wilder.
- Variance = 0.0004: numbers are typically around ± 0.02 . Almost flat.

When we say “we want variance = 1,” we mean: *keep the numbers in a moderate range where nothing explodes or vanishes.*

Variance in our Proust model

Our embedding matrix is initialized with `np.random.randn(...)*0.02`, which means:

$$\text{Each element} \sim \mathcal{N}(0, 0.02^2) = \mathcal{N}(0, 0.0004)$$

So each embedding element is typically ± 0.02 . Very small on purpose — to prevent activations from exploding through the network layers on the very first forward pass.

3 The Core Theorem: Variance of a Sum

This is the single most important result for understanding $\sqrt{d}$. Everything else follows from it.

Theorem: Variance of a sum of independent variables

Let $X_1, X_2, \dots, X_d$ be **independent** random variables, each with mean 0 and variance σ^2 . Then:

$$S = X_1 + X_2 + \dots + X_d$$

has:

$$\mathbb{E}[S] = 0, \quad \text{Var}(S) = d \cdot \sigma^2$$

Proof: Since the variables are independent, variances add:

$$\begin{aligned} \text{Var}(S) &= \text{Var}(X_1) + \text{Var}(X_2) + \dots + \text{Var}(X_d) \\ &= \sigma^2 + \sigma^2 + \dots + \sigma^2 \\ &= d \cdot \sigma^2 \end{aligned}$$

Standard deviation of the sum: $\sigma_S = \sqrt{d \cdot \sigma^2} = \sqrt{d} \cdot \sigma$.

Why sums get bigger

Imagine flipping a coin that says +1 or -1. After 1 flip, you could be at ± 1 . After 4 flips? You might be at +2 or -2 (not ± 4). After 64 flips? Typically around ± 8 .

The pattern: after d flips, you're typically at $\pm\sqrt{d}$.

This is the **random walk**: each step is random, but the accumulation grows as $\sqrt{d}$, not as d . It's a fundamental law of probability.

Number of terms (d)	$\sqrt{d}$	Typical magnitude of sum
1	1.0	± 1
4	2.0	± 2
16	4.0	± 4
64	8.0	± 8
512	22.6	± 23

The sum of d independent random numbers has standard deviation $\sqrt{d}$ times bigger than each individual number. This is why $\sqrt{d}$ is the natural correction factor.

In our 64-dimensional model

Each embedding vector has $d_{\text{model}} = 64$ elements. If each element has $\sigma = 0.02$, then any operation that sums all 64 elements produces a result with:

$$\sigma_{\text{sum}} = \sqrt{64} \times 0.02 = 8 \times 0.02 = 0.16$$

The sum is $8\times$ bigger than each element. That factor of 8 is $\sqrt{d_{\text{model}}}$.

4 Dot Products: Where $\sqrt{d}$ Becomes Critical

The dot product is the most important operation in transformers. Attention scores, embedding lookups, and linear projections all reduce to dot products. And dot products suffer directly from the variance accumulation we just described.

Theorem: Variance of a dot product

Let $\mathbf{q} = (q_1, \dots, q_d)$ and $\mathbf{k} = (k_1, \dots, k_d)$ be two random vectors where all elements are **independent** with mean 0 and variance σ^2 .

The dot product $\mathbf{q} \cdot \mathbf{k} = \sum_{i=1}^d q_i k_i$ has:

$$\mathbb{E}[\mathbf{q} \cdot \mathbf{k}] = 0$$

$$\text{Var}(\mathbf{q} \cdot \mathbf{k}) = d \cdot \sigma^4$$

Proof:

Each term of the sum is $Z_i = q_i \cdot k_i$.

Step 1: Mean of each term.

Since q_i and k_i are independent and both have mean 0:

$$\mathbb{E}[q_i \cdot k_i] = \mathbb{E}[q_i] \cdot \mathbb{E}[k_i] = 0 \cdot 0 = 0$$

Step 2: Variance of each term.

For independent zero-mean variables:

$$\text{Var}(q_i \cdot k_i) = \mathbb{E}[q_i^2 \cdot k_i^2] - (\mathbb{E}[q_i \cdot k_i])^2 = \mathbb{E}[q_i^2] \cdot \mathbb{E}[k_i^2] - 0 = \sigma^2 \cdot \sigma^2 = \sigma^4$$

Step 3: Variance of the sum.

The d terms $Z_1, \dots, Z_d$ are independent (since different indices are independent), so:

$$\text{Var}\left(\sum_{i=1}^d Z_i\right) = d \cdot \sigma^4$$

Standard deviation: $\sqrt{d} \cdot \sigma^2$.

What this means in plain language

A dot product sums d terms. Each term is small (a product of two small numbers), but there are d of them. By the “variance of sums” theorem, the result is $\sqrt{d}$ times bigger than any single term.

Concrete example with $d = 64$ and $\sigma = 1$:

Each term $q_i \cdot k_i$ has variance $1^4 = 1$. The sum of 64 such terms has variance $64 \cdot 1 = 64$, so $\sigma_{\text{dot}} = \sqrt{64} = 8$.

This means dot product values are typically around ± 8 — not ± 1 .

The critical question: if we want the dot product to have variance 1 (so that downstream operations like softmax work well), how do we fix this?

Answer: divide by $\sqrt{d}$:

$$\text{Var}\left(\frac{\mathbf{q} \cdot \mathbf{k}}{\sqrt{d}}\right) = \frac{\text{Var}(\mathbf{q} \cdot \mathbf{k})}{d} = \frac{d \cdot \sigma^4}{d} = \sigma^4$$

For $\sigma = 1$: variance becomes 1. The scaling perfectly compensates.

In our attention mechanism (src/attention.py:224)

Our multi-head attention computes:

```
scores = Q @ K.transpose(0, 1, 3, 2)    # (1, 2, 4, 32) @ (1, 2, 32, 4) = (1, 2, 4, 4)
scores = scores / np.sqrt(self.d_k)       # divide by sqrt(32) = 5.66
```

Here $d_k = 32$ (per-head dimension). Each attention score is a dot product of 32-dimensional query and key vectors. Without dividing by $\sqrt{32} \approx 5.66$, the scores would have standard deviation ~ 5.66 (assuming unit-variance elements), making softmax produce near-one-hot outputs.

5 Why Softmax Cares About Scale

The softmax function

For a vector $\mathbf{z} = (z_1, \dots, z_n)$:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

The output is a probability distribution: all values are in $[0, 1]$ and sum to 1.

Key property: softmax is sensitive to the *magnitude* of its inputs.

- Small inputs $\mathbf{z} \approx (0.1, 0.2, 0.1)$: output $\approx (0.32, 0.36, 0.32)$ — uniform, spread out.
- Large inputs $\mathbf{z} \approx (1, 20, 1)$: output $\approx (0.0, 1.0, 0.0)$ — one-hot, all mass on one element.

Softmax as a “temperature dial”

Think of softmax as a dial between two extremes:

Inputs near zero	$\longleftrightarrow$	Inputs very large
Uniform distribution		One-hot (argmax)
“Consider everything equally”		“Focus on one thing only”
Gradients are healthy		Gradients are near zero

When attention scores are too large (because we didn’t scale), softmax becomes a hard argmax. The output says “attend 100% to position 3, 0% everywhere else.” The gradient of this is essentially zero for all inputs except the maximum, so **the model cannot learn to adjust its attention pattern**.

This is called **softmax saturation**: the function is “stuck” at the extremes.

Numerical example with our $d_k = 32$

Suppose $\mathbf{q}$ and $\mathbf{k}$ are 32-dimensional with elements $\sim \mathcal{N}(0, 1)$.

Without scaling:

$$\text{score} = \mathbf{q} \cdot \mathbf{k} \Rightarrow \sigma_{\text{score}} = \sqrt{32} \approx 5.66$$

A typical score vector might be: [2.1, 15.3, -4.7, 8.2].

$\text{softmax}([2.1, 15.3, -4.7, 8.2]) \approx [0.000, 0.999, 0.000, 0.001]$

Almost one-hot. Gradient ≈ 0 for all but one position.

With scaling by $\sqrt{32}$:

$$\text{score} = \frac{\mathbf{q} \cdot \mathbf{k}}{\sqrt{32}} \Rightarrow \sigma_{\text{score}} = 1$$

A typical score vector might be: [0.4, 2.7, -0.8, 1.4].

$\text{softmax}([0.4, 2.7, -0.8, 1.4]) \approx [0.06, 0.60, 0.02, 0.17]$

Smooth distribution. All positions get meaningful gradient. The model can learn.

6 The Other $\sqrt{d}$: Embedding Scaling

Now we connect to the embedding layer. This is a *different* appearance of $\sqrt{d}$, but it comes from the same mathematical root.

Embedding scaling in the paper

Section 3.4 of “Attention Is All You Need” (Vaswani et al., 2017):

“In the embedding layers, we multiply those weights by $\sqrt{d_{\text{model}}}$.”

The embedding forward pass is:

$$\text{output} = \underbrace{\mathbf{E}[\text{token_ids}] \cdot \sqrt{d_{\text{model}}}}_{\text{scaled token embedding}} + \underbrace{\mathbf{PE}[\text{position}]}_{\text{positional encoding}}$$

where $\mathbf{E}$ is the learned embedding matrix and $\mathbf{PE}$ is the fixed sinusoidal encoding.

Why embeddings need scaling: the magnitude mismatch

Two signals are being **added** together in the embedding layer:

1. **Token embedding**: what character is this? (learned)
2. **Positional encoding**: where is it in the sequence? (fixed, $\sin/\cos$)

For addition to be meaningful, both signals must have **comparable magnitudes**. If one is $50\times$ bigger, the other is effectively invisible in the sum.

The positional encoding uses $\sin$ and $\cos$, which produce values in $[-1, +1]$. Each element has a typical absolute value around 0.7 (the RMS of a sine wave is $1/\sqrt{2}$).

The token embedding is initialized small. How small depends on the scheme:

Initialization	Per-element σ	Vector L2 norm ($\approx \sqrt{d} \cdot \sigma$)
Our NumPy ($\times 0.02$)	0.02	$\sqrt{64} \times 0.02 = 0.16$
Paper ($\sim \mathcal{N}(0, 1/\sqrt{d})$)	0.125	$\sqrt{64} \times 0.125 = 1.0$
PyTorch <code>nn.Embedding</code> default	1.0	$\sqrt{64} \times 1.0 = 8.0$
Positional encoding	~ 0.7	$\sqrt{64} \times 0.7 = 5.6$

Without scaling, our NumPy embeddings have norm 0.16 vs positional encoding norm 5.6. The ratio is $35 : 1$ — position completely drowns out content.

After multiplying by $\sqrt{d_{\text{model}}} = \sqrt{64} = 8$:

$$\text{Scaled embedding norm} = 8 \times 0.16 = 1.28$$

Now the ratio is $5.6 : 1.28 \approx 4.4 : 1$. Not perfectly balanced, but the content signal is now **audible**. During training, the embedding weights grow and the balance improves further.

In our code: `src/embedding.py:95-100`

```
# Step 1: Token embedding lookup
token_emb = self.token_embedding[token_ids]          # (batch, seq, 64)

# Scale embeddings by sqrt(d_model) as in original paper
token_emb = token_emb * np.sqrt(self.d_model)       # multiply by 8.0

# Step 2: Add positional encoding
pos_emb = self.pos_encoding[:seq_len, :]           # (seq, 64)
output = token_emb + pos_emb                        # (batch, seq, 64)
```

Before our bug fix, the `np.sqrt(self.d_model)` line was missing. The PyTorch version (`src/model_torch.py:145`) always had it:

```
token_emb = token_emb * math.sqrt(self.d_model)     # multiply by 8.0
```

Dimension Check

Stage	Shape	Per-element magnitude
Token IDs	(batch, seq)	integers in [0, 95]
Raw embedding lookup	(batch, seq, 64)	~ 0.02
After $\times \sqrt{64}$	(batch, seq, 64)	~ 0.16
Positional encoding	(seq, 64)	~ 0.7
Sum (embedding + positional)	(batch, seq, 64)	$\sim 0.7\text{--}0.9$

7 Unifying the Two Scalings

The two $\sqrt{d}$ scalings in a transformer

1. **Embedding layer:** multiply by $\sqrt{d_{\text{model}}}$
Purpose: scale UP embeddings to match positional encoding magnitude.
2. **Attention layer:** divide by $\sqrt{d_k}$
Purpose: scale DOWN dot products to prevent softmax saturation.

Both arise from the same mathematical fact:

$$\text{Var}\left(\sum_{i=1}^d X_i\right) = d \cdot \text{Var}(X_i)$$

When you sum d random terms, the result has standard deviation $\sqrt{d}$ times the standard deviation of each term. The correction factor is always $\sqrt{d}$.

One formula, two directions

The $\sqrt{d}$ factor is like a “dimension tax” — the price you pay for working in high dimensions.

- In **embeddings**, we PAY the tax voluntarily: multiply by $\sqrt{d}$ to boost the signal up to where positional encodings live. We *want* the embedding to be bigger.
- In **attention**, we UNDO the tax: divide by $\sqrt{d}$ to bring the dot product back down to a safe range for softmax. We *don't want* the score to be bigger.

Same $\sqrt{d}$, opposite directions, same reason.

Full signal flow through our Proust model

Let us trace the magnitude of a single value through the entire architecture:

Stage	Code location	Per-element σ
Embedding init	<code>embedding.py:38</code>	0.02
$\times \sqrt{64}$ scaling	<code>embedding.py:100</code>	$0.02 \times 8 = 0.16$
+ positional encoding	<code>embedding.py:109</code>	~ 0.7
$\rightarrow$ into attention	<code>attention.py:199</code>	~ 0.7
Q, K projection ($\times W$)	<code>attention.py:199</code>	$\sqrt{64} \times 0.7 \times 0.02 \approx 0.11$
Dot product $Q \cdot K$	<code>attention.py:221</code>	$\sqrt{32} \times 0.11^2 \approx 0.07$
$\div \sqrt{32}$ scaling	<code>attention.py:224</code>	$0.07/5.66 \approx 0.01$
Softmax	<code>attention.py:240</code>	probabilities in $[0, 1]$

At every stage, the magnitudes stay in a “reasonable” range. No explosions, no vanishing. The two $\sqrt{d}$ scalings are two of the key mechanisms that ensure this.

8 The Deeper Connection: L2 Norm and Dimension

L2 norm of a random vector

For a d -dimensional vector $\mathbf{v}$ where each element v_i has mean 0 and variance σ^2 :

$$\|\mathbf{v}\|^2 = \sum_{i=1}^d v_i^2$$

Taking the expected value:

$$\mathbb{E}[\|\mathbf{v}\|^2] = \sum_{i=1}^d \mathbb{E}[v_i^2] = d \cdot \sigma^2$$

So:

$$\|\mathbf{v}\| \approx \sqrt{d} \cdot \sigma$$

The L2 norm of a random vector scales as $\sqrt{d}$ — same factor, same reason.

High dimensions are strange

In 2D, a vector with elements $\sim \mathcal{N}(0, 1)$ has length $\approx \sqrt{2} \approx 1.4$. In 64D, the same distribution gives length $\approx \sqrt{64} = 8$. In 768D (GPT-2), it's $\approx \sqrt{768} \approx 27.7$.

High-dimensional vectors are **much longer** than you'd expect from their individual elements. This is not because any single element is large — it's because there are *many small contributions* that accumulate via the Pythagorean theorem.

This is why $\sqrt{d}$ appears so often in machine learning:

- Xavier/Glorot initialization: $\sigma = 1/\sqrt{d_{\text{in}}}$
- Attention scaling: $\div \sqrt{d_k}$
- Embedding scaling: $\times \sqrt{d_{\text{model}}}$
- Layer normalization: normalize the $\sqrt{d}$ -sized vector back to unit variance

They are all compensating for the same geometric reality of high dimensions.

Numerical check with our model

Our embedding vectors: $d = 64$, $\sigma = 0.02$:

$$\|\mathbf{e}\| \approx \sqrt{64} \times 0.02 = 0.16$$

Our positional encoding vectors: $d = 64$, elements $\in [-1, 1]$ with RMS ≈ 0.7 :

$$\|\mathbf{p}\| \approx \sqrt{64} \times 0.7 = 5.6$$

Ratio without scaling: $5.6/0.16 = 35$.

After $\times \sqrt{64}$ scaling: $5.6/(0.16 \times 8) = 5.6/1.28 = 4.4$.

Much better — and training will close the remaining gap.

9 Summary Table

Where	Formula	Dimension	Why
Embedding layer	$\mathbf{e} \times \sqrt{d_{\text{model}}}$	$d_{\text{model}} = 64$	Scale up to match positional encoding magnitude
Attention scores	$\mathbf{q} \cdot \mathbf{k} / \sqrt{d_k}$	$d_k = 32$	Scale down to prevent softmax saturation
Xavier init	$\sigma = 1/\sqrt{d_{\text{in}}}$	$d_{\text{in}} = \text{input dim}$	Keep layer output variance $\approx$ input variance
LayerNorm	Normalize to $\mu = 0, \sigma = 1$ across d_{model}	$d_{\text{model}} = 64$	Undo any accumulated magnitude drift

All four are different manifestations of the same underlying principle: **when you sum or combine d independent numbers, the result grows by $\sqrt{d}$, and you must compensate.**

10 Key Identities and Properties

1. $\text{Var}(aX) = a^2 \cdot \text{Var}(X)$ (scaling by constant a)
2. $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$ (if X, Y independent)
3. $\text{Var}(X \cdot Y) = \text{Var}(X) \cdot \text{Var}(Y)$ (if X, Y independent, zero-mean)
4. $\text{Var}\left(\sum_{i=1}^d X_i\right) = d \cdot \sigma^2$ (i.i.d. variables, the core theorem)
5. $\text{Var}\left(\frac{1}{\sqrt{d}} \sum_{i=1}^d X_i\right) = \sigma^2$ (the fix: divide by $\sqrt{d}$)
6. $\|\mathbf{v}\| \approx \sqrt{d} \cdot \sigma$ (L2 norm of a random d -dimensional vector)

Identities 4 and 5 are the entire story. Everything else is a special case.

11 What This Document Does Not Cover (Yet)

- **Gradient flow**: how $\sqrt{d}$ affects the backward pass and gradient magnitudes (related, but requires chain rule derivations).
- **Pre-norm vs post-norm**: how the placement of LayerNorm interacts with these scaling choices.
- **RoPE and ALiBi**: modern positional encodings that avoid the additive approach entirely.
- **FlashAttention**: how fused kernels handle the scaling internally.
- **Non-unit-variance initializations**: what happens when the i.i.d. assumption breaks (e.g., after several layers of training when weights are no longer near initialization).

*“The real voyage of discovery consists not in seeking new landscapes,
but in having new eyes.”*

— Marcel Proust, *La Prisonnière*